3. NF_INET_FORWARD – Packets not addressed to the local machine (hence forwarded) will pass this hook.
4. NF_INET_LOCAL_OUT – Outgoing packets generated by processes on the local machine pass this hook.
5. NF_INET_POST_ROUTING—Any packet, locally generated or forwarded, is accessible here for the last time just before it exits out of the system and hits the wire.

At each of these hooks, Netfilter checks if any kernel modules have been registered. If so, then the packet is given to the kernel modules. The kernel module will invoke a call-back function called the hook function that takes charge of the packet and processes it (inspects, filters, alters, etc). When the call-back function is done with packet processing, it will instruct Netfilter what to do with the packet by returning the following return codes.

NF_DROP – Netfilter drops the packet and the resources offered to the packet are de-allocated by the kernel.

NF_ACCEPT – The packet continues its standard network stack traversal.

NF_STOLEN – Instructs Netfilter to drop the processing of the packet. The hook function owns the responsibility of the packet and should take care of de-allocating the resources.

NF_QUEUE - Netfilter queues the packet for user space programs.

NF_REPEAT – Netfilter calls the hook function again.

Getting hands-on: Implementing a simple firewall

Let's reinforce our understanding of Netfilter by writing a simple firewall using the features provided by the Netfilter framework. Our firewall will be a loadable kernel module (accompanied with a Makefile), which hooks at the *NF_IP_PREROUTING* hook. The firewall drops all the *ICMP* ping packets from the machine with the source address 192.168.164.51. Along the way, let's familiarise ourselves with Netfilter's specific data structures, functions and macros, as we encounter them.

The implementation involves three steps:

1. Defining a hook function of type *nf_hookfn* that contains packet processing and manipulation logic.
2. Initialising the *nf_hook_ops* structure with the hook function, the protocol, hook number and the priority.
3. Registering the fully initialised structure *nf_hook_ops* in the *init* function of the kernel module.

1) Defining a hook function: The skeleton for our firewall implementation is the kernel module file *icmp_drop.c* . Inside the module, we need to define a call-back function - a hook function that inspects and drops the ICMP packets from the specific IP address. The hook functions are of type *nf_hookfn* and their prototype is defined in *<linux/netfilter.h>* as follows:

```
typedef unsigned int nf_hookfn(unsigned int hooknum,
                               struct sk_buff *skb,
                               const struct net_device *in,
                               const struct net_device *out,
                               int (*okfn)(struct sk_buff
*));
```

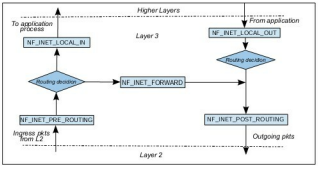

Figure 1: Netfilter hooks

The following list describes the *nf_hookfn* arguments:

hooknum - indicates one of the five earlier-described hooks.

***Skb* –** is a pointer to the network packet buffer.

in and ***out*** - points to the network device structure of Linux. It points to the network interface packet is received at the hook and for the interface packet is passed to. However, depending upon the hook at which the packet is traversing, in and out can be *NULL*.

okfn - is the function that gets called when the hook function(s) registered at the hook returns *NF_ACCEPT*.

Our hook function that's called *hook_fn_icmp* is defined from Line 23 to 38. At Lines 31 and 32, packets matching the IP address and ICMP protocol (0x01) are dropped by returning *NF_DROP* at Line 35. The rest of the unmatched packets are returned to Netfilter by returning *NF_ACCEPT* at Line 37. We also print the dropped packet count visible in kernel logs via *dmesg*.

2) Initialising the nf_hook_ops structure: Once we have the hook function ready, we must initialise the *nf_hooks_ops* structure whose prototype is defined in *<linux/netfilter.h>*

```
struct nf_hook_ops
{
     struct list_head list;
     /* User fills in from here down. */
     nf_hookfn *hook;
     int pf;

     int hooknum;
     /* Hooks are ordered in ascending priority. */
int priority;
};
```

The structure members are shown below:

***list* –** is the linked list of the *nf_hook_ops* structure maintained by the kernel.